

$$1.1 \quad p' = (x_1, \dots, x_N, \omega) = \left(\frac{x_1}{\omega}, \dots, \frac{x_N}{\omega}, 1\right) \Rightarrow x'_i = \frac{x_i}{\omega} \quad (1 \leq i \leq N)$$

① 在将相机与真实世界坐标对齐时, 需要进行平移

$$\begin{cases} X_1 = X + t_x \\ Y_1 = Y + t_y \\ Z_1 = Z + t_z \end{cases} \quad \text{表达这个式子原本需要矩阵相加} \quad \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

但引入齐次坐标后可化为矩阵相乘

$$\begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

这样内、外参矩阵可直接全乘起来, 形式统一

② 投影到相机坐标系时 $\begin{cases} x = \frac{fX}{Z} \\ y = \frac{fY}{Z} \end{cases}$, 要除以 Z , 这不是线性变换

不能用矩阵表示, 但引入二维齐次坐标后可写成

$$\begin{bmatrix} f & c_x \\ f & c_y \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX + c_x Z \\ fY + c_y Z \\ Z \end{bmatrix} = \begin{bmatrix} \frac{fX}{Z} + c_x \\ \frac{fY}{Z} + c_y \\ 1 \end{bmatrix}$$

这样也可以化为矩阵相乘, 使形式统一, 计算方便

③ 在表示一条直线上的无穷远点时齐次坐标可表示为

$$\lim_{t \rightarrow \infty} \begin{bmatrix} x_0 + td_x \\ y_0 + td_y \\ z_0 + td_z \\ 1 \end{bmatrix} = \lim_{t \rightarrow \infty} \begin{bmatrix} \frac{x_0}{t} + d_x \\ \frac{y_0}{t} + d_y \\ \frac{z_0}{t} + d_z \\ \frac{1}{t} \end{bmatrix} = \begin{bmatrix} d_x \\ d_y \\ d_z \\ 0 \end{bmatrix}$$

这样形式与一般点相同, 形式统一.

1.2 一方面, 当相机远离物体时, 物体大小本应该减小, 但此时若增大焦距, 由 $\tan \frac{\alpha}{2} = \frac{d}{2z}$, 与 $z' = (1+m)f$

知 α 变小, 于是 FOV 变小, 造成物体大小增大. 二者协调

适当可使物体在成像平面上大小不变, 且达到背景变远的^(w)

效果 (即 foreground dominates surroundings)

另一方面, 相机靠近物体且减小焦距, 二者协调适当可使背景
变近(大). 物体大小不变 (即 surrounding overwhelm foreground)



2. Vanishing Line 如图中深黄色线所示(平行地面)

在图中 vanishing line 高度与 A 点高度比例为 2.06:1

因此相机高度 $H \approx 2.06 \times 0.76 = 1.57$ 米

真实世界中 $A(0.374, 0.76)$, 相机 $(13.445, 1.57)$

A 点相对于相机的坐标为 $(-13.4, -0.76, -0.81) = (x, y, z)$

经测量得 A 点在照片中坐标为 $(1857, 1119) = (y, z)$

而 $w \begin{bmatrix} y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} f & C_y \\ & f & C_z \\ & & 1 \end{bmatrix} \begin{bmatrix} Y \\ Z \\ X \end{bmatrix}$ ← 这个是 $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ 乘以了旋转矩阵后的

$$\begin{bmatrix} wy \\ wz \\ w \end{bmatrix} = \begin{bmatrix} fY + C_y X \\ fZ + C_z X \\ X \end{bmatrix} \Rightarrow \frac{y}{z} = \frac{fY + C_y X}{fZ + C_z X} \quad \begin{matrix} C_y = 2048 \\ C_z = 1536 \end{matrix}$$

$$\frac{1857}{1119} = \frac{-0.76f - 13.4 \times 2048}{-0.81f - 13.4 \times 1536} \Rightarrow f = 5382 \quad (\text{像素})$$

内参矩阵为 $\begin{bmatrix} 5382 & 0 & 2048 \\ 0 & 5382 & 1536 \\ 0 & 0 & 1 \end{bmatrix}$

3. 启动方式为 `python gaussian_pyramid.py`

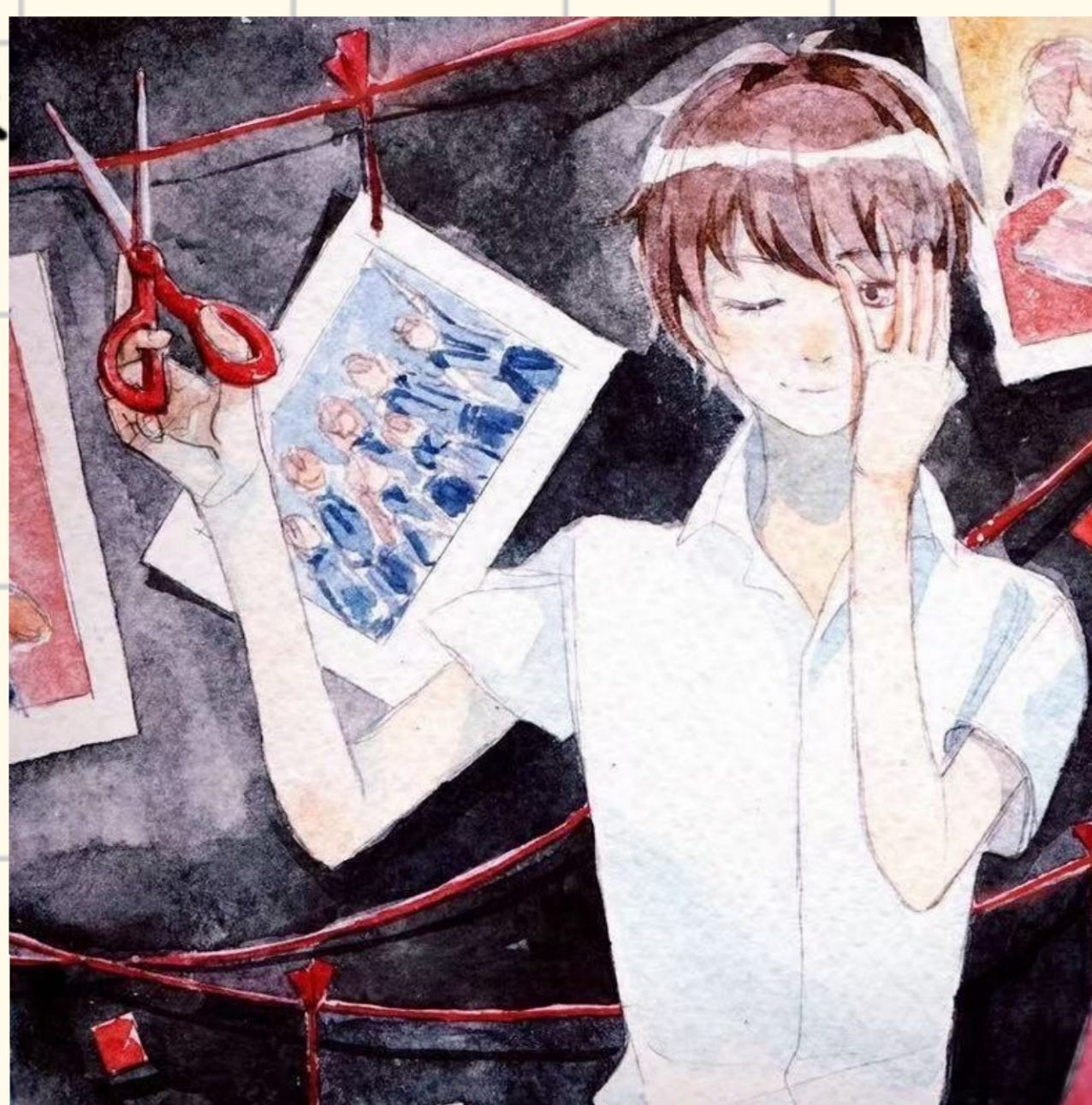
(可选 `--f` 用于指定函数, `--input` 用于指定

输入图片, `--sigma` 用于指定 σ . 如

`python gaussian_pyramid.py --input frog.jpg`

`--f low-pass --sigma 5.0`)

用本人微信头像
做测试, 效果
如右图:



美中不足是由于 `pad` 是
用 0 补齐, 所以生成的图片会有黑色边边
暂时能想到的方法
只有每次裁掉最外面一圈